1. (2 Punkte) Was wird ausgegeben?

```
a = ['banane', 'Apfel', 'kiwi', 'Birne']
print(sorted(a))
print(sorted(a, key=str.lower))
```

Lösung:

```
['Apfel', 'Birne', 'banane', 'kiwi']
['Apfel', 'banane', 'Birne', 'kiwi']
```

Großbuchstaben haben kleinere Unicode-Werte als Kleinbuchstaben und stehen deshalb vorne.

2. (2 Punkte) Was wird ausgegeben?

```
a = ['Ben', 'Anna', 'Tim', 'Eva', 'Lena']
print(sorted(a, key=len))
print(sorted(a, key=lambda x: (len(x), x)))
```

Lösung:

```
['Ben', 'Tim', 'Eva', 'Anna', 'Lena']
['Ben', 'Eva', 'Tim', 'Anna', 'Lena']
```

Das Sortieren ist stabil: Bei gleicher Länge bleibt in der ersten Zeile die ursprüngliche Reihenfolge erhalten.

3. (2 Punkte) Gegeben sei die Tupelliste `a = [(3,5),(7,3),(1,2),(10,1)]`. Erzeuge mit dem lambda-Operator zwei Listen b1 und b2. b1 ist sortiert nach der Summe der Komponenten, b2 ist absteigend nach der 2. Komponente sortiert.

Lösung:

```
b1 = sorted(a,key = lambda x: x[0]+x[1])
b2 = sorted(a,key = lambda x: x[1], reverse=True)
```

4. (2 Punkte) Gegeben sei die Tupelliste `a = [(3,5,4),(7,3,2),(1,2,1),(10,1,6)]`.
Erzeuge mit dem lambda-Operator zwei Listen b1 und b2. b1 ist sortiert nach dem Produkt der drei Komponenten, b2 ist absteigend nach der letzten Komponente sortiert.

Lösung:

```
b1 = sorted(a,key = lambda x: x[0]*x[1]*x[2])
b2 = sorted(a,key = lambda x: x[2], reverse=True)
```

5. (3 Punkte) Gegeben sei die Liste:

```
a = []
a.append("Emma Elsenpeter aus Willegassen")
a.append("Emilia Poggenpohl aus Mastholte")
a.append("Clara Mörtenkötter aus Haddenhausen")
a.append("Noah Schniggendiller aus Barntrup")
a.append("Ben Trendelberend aus Brakelsiek")
a.append("Leon Eschengerd aus Altendonop")
```

Schreibe jeweils eine Zeile um die Liste a zu sortieren nach
- Vorname
- Nachname
- Ort (abwärts)

Lösung:

```
a.sort(key = lambda x: x.split()[0])
a.sort(key = lambda x: x.split()[1])
a.sort(key = lambda x: x.split()[3],reverse=True)
```

6. (2 Punkte) Gegeben sei die Liste `a = [2301,5401,1303,6403,4302,2402]`.

Sortiere die Liste mit einer Zeile nach den 100er-Zahlen. Das Ergebnis sieht dann so aus:

`2301 1303 4302 5401 6403 2402`.

Lösung:

```
a.sort(key = lambda x: (x%1000)//100)
```

7. (2 Punkte) Gegeben sei eine Liste a mit positiven ganzen Zahlen. Sortiere die Liste abwärts nach der Quersumme der Zahlen.

Lösung:

```
a.sort(key = lambda x: sum([int(k) for k in str(x)]),reverse=True)
```

8. (2 Punkte) Gegeben sei eine Liste `a` mit Strings.

Sortiere die Liste aufsteigend nach der Länge des Strings. Bei gleicher Länge sortiere alphabetisch.

Lösung:

```
a.sort(key = lambda x: (len(x),x))
```

9. (3 Punkte) Das Dict `h` enthält, wie oft Wörter in einem Text vorkommen:

```
h = {'der': 5, 'Hund': 2, 'bellt': 2, 'sehr': 2, 'laut': 1}
```

   (a) (2 Punkte) Erzeuge mit einer Zeile eine Liste der Wörter, absteigend sortiert nach der Häufigkeit. Bei gleicher Häufigkeit sollen die Wörter alphabetisch sortiert sein.

   (b) (1 Punkt) Wie sieht die Liste aus?

Lösung:

```
woerter = sorted(h, key=lambda w: (-h[w], w))
```

b) `['der', 'Hund', 'bellt', 'sehr', 'laut']`

10. (3 Punkte) Gegeben sei das Rucksackproblem mit folgenden Eingaben:

```
wert = [1,11,13,9]
gewicht = [2,6,8,4]
kapazitaet = 10
```

Benenne die im Unterricht besprochenen drei Greedy-Strategien und ermittle die Indizes der genommenen Items, den Wert und das Gewicht des Rucksacks für jeden der drei Fälle.

Lösung:

```
Die wertvollsten zuerst: Items = 0 2 , Wert = 14 , Gewicht = 10
Die leichtesten zuerst: Items = 0 3 , Wert = 10 , Gewicht = 6
Die wertdichtesten zuerst: Items = 1 3 , Wert = 20 , Gewicht = 10
```

11. (3 Punkte) Gegeben ist das Rucksackproblem mit folgenden Eingaben:

```
wert = [10, 7, 7, 1]
gewicht = [6, 5, 5, 1]
kapazitaet = 10
```

   (a) (2 Punkte) Ermittle für die drei Greedy-Strategien aus dem Unterricht die Indizes der genommenen Items und den Wert des Rucksacks.

   (b) (1 Punkt) Welche Auswahl ist die beste? Was folgt daraus für Greedy-Verfahren?

Lösung:

```
a) Die wertvollsten zuerst:   Items 0 3, Wert 11
   Die leichtesten zuerst:    Items 1 3, Wert 8
   Die wertdichtesten zuerst: Items 0 3, Wert 11
```

b) Items 1 und 2 mit Wert 14 und Gewicht 10. Ein Greedy-Verfahren findet nicht immer die beste Lösung.